Отчет по отладке приложения

Григорян Александр Радикович

Шаг 1: Анализ и первый запуск (исправление бага №1)

- **Что сделал**: распаковал архив, выполнил docker compose up –build
- **Проблема**: контейнер app-1 падал сразу на команде alembic upgrade head с ошибкой Pydantic:
- **Файл и строка:** app/core/config.py (класс Settings). Исправил опечатку: было DATABSE

Рисунок 1

Шаг 2: Исправление бага №2 (bash в контейнере)

- **Что сделал:** Повторил запуск после предыдущего фикса.
- **Проблема:** Команда bash -c падала — bash: command not found.
- **Файл и строка:** docker-compose.yaml (блок app.command).
- **Код до:** bash -c "alembic upgrade head && uvicorn ..."
- **Код после:** sh -c "alembic upgrade head && uvicorn ..."

- **Причина:** В образе python:3.11-slim утилита bash отсутствует.

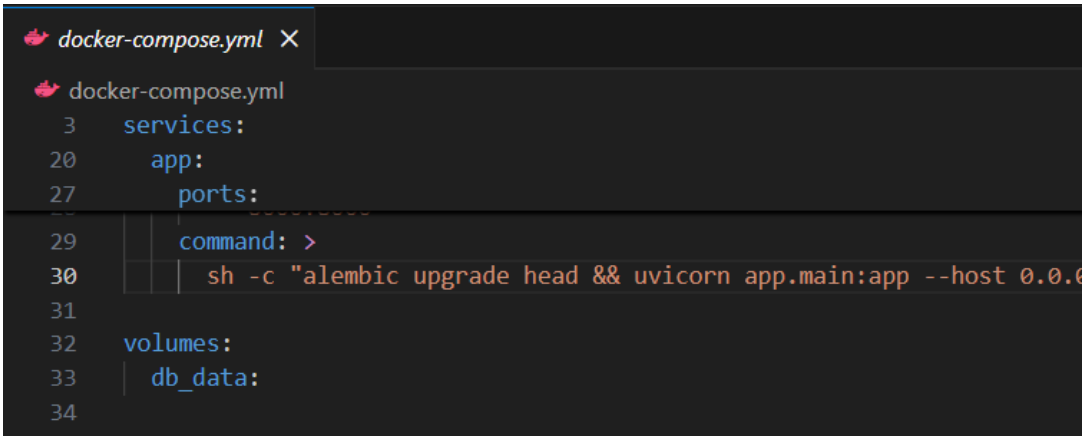

Рисунок 2

Шаг 3: Исправление бага №3 (Alembic)

- **Что сделал:** После запуска увидел, что миграции не применяются.
- **Проблема:** Неправильный sqlalchemy.url в конфиге Alembic.
- **Файл и строка:**alembic.ini.
- **Код до:**sqlalchemy.url = driver://user:pass@localhost/dbname
- **Код после:**sqlalchemy.url = postgresql+asyncpg://postgres:postgres@db:5432/postgres
- **Причина:** Стандартный шаблон Alembic не был адаптирован под проект.

Шаг 4: Исправление бага №4, 5 (Dockerfile — вложенность)

- **Что сделал:** Проверил структуру после сборки.
- **Проблема:** Лишняя вложенность папок из-за неправильных COPY.
- **Файл и строка:** Dockerfile (строки 8 и 11).
- **Код до:**COPY requirements.txt /app/ и COPY . /app/
- **Код после:**COPY requirements.txt . и COPY . .

- **Причина:** Создавалась структура /app/app/app/....

```
Dockerfile
Dockerfile > FROM
1   FROM python:3.11-slim
2
3   WORKDIR /app
4
5   ENV PYTHONDONTWRITEBYTECODE=1 \
6       PYTHONUNBUFFERED=1
7
8   COPY requirements.txt .
9   RUN pip install --no-cache-dir -r requirements.txt
10
11  COPY . /app/
12
13  CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Рисунок 3

Шаг 5: Исправление бага №6 (requirements.txt)

- **Что сделал:** Проверил зависимости.
- **Проблема:** Строки fastapi=999.0.0 и python_version < "3.8" (неработающие версии).
- **Файл и строка:**requirements.txt.
- **Код до:**fastapi=999.0.0; python_version < "3.8"
- **Код после:** Удалил эти строки (оставил только fastapi без версии).
- **Причина:** Дублирование с Dockerfile и несуществующая версия.

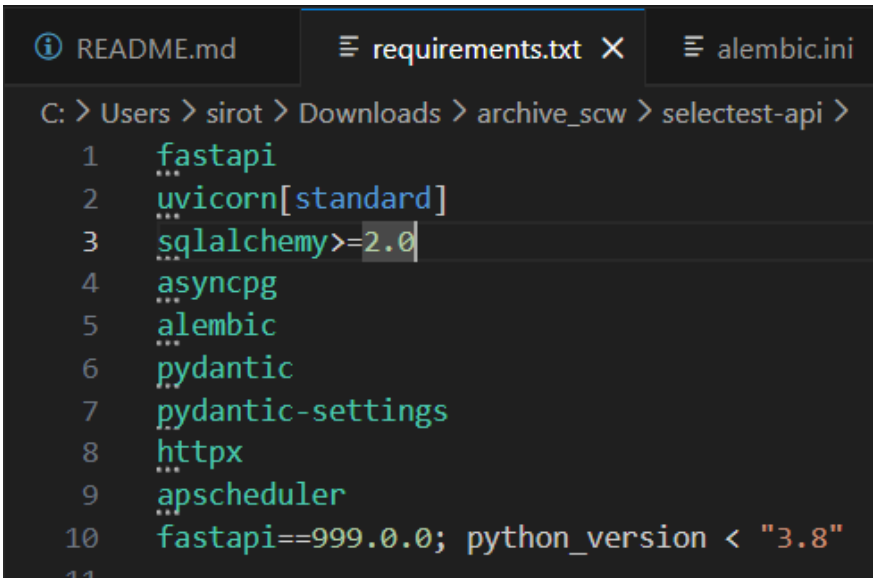

Рисунок 4

Шаг 6: Исправление бага №7 (healthcheck и мелкие правки в compose)

- **Что сделал:** Улучшил стабильность запуска.
- **Проблема:** Healthcheck был неполным.
- **Файл и строка:**docker-compose.yaml.
- **Код до:**test: ["CMD-SHELL", "pg_isready -U postgres"]
- **Код после:** Добавил -d postgres и увеличил интервал.
- **Причина:** Postgres иногда не успевал подняться.

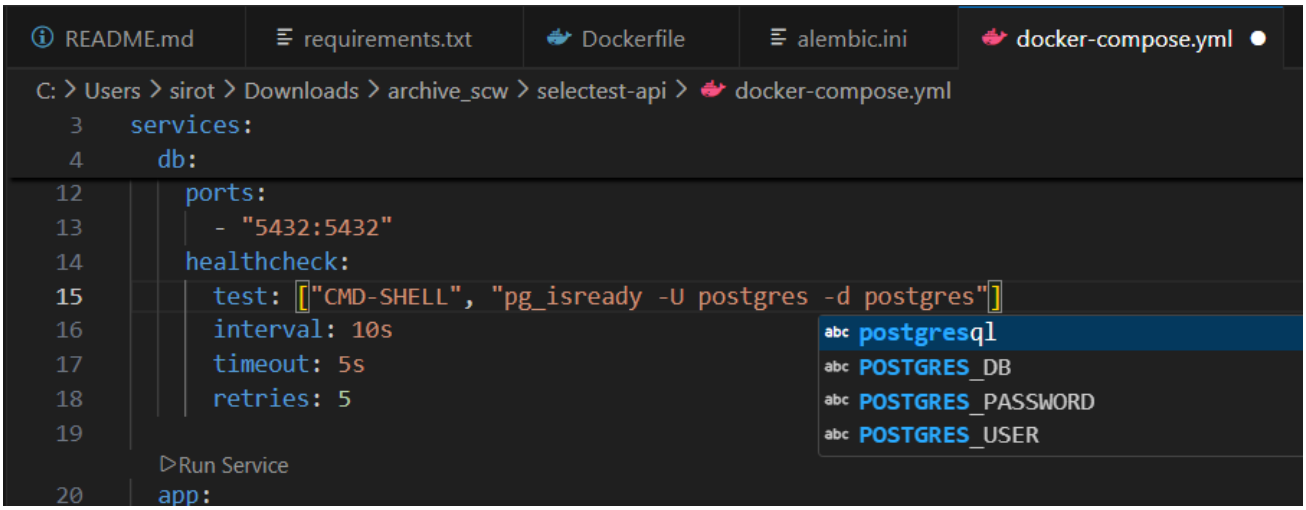

Рисунок 5

Шаг 7: Исправление последнего бага №8 (парсер)

- **Что сделал:** Приложение запустилось, но падал парсинг каждые 5 минут.
- **Проблема:** AttributeError: 'NoneType' object has no attribute 'name'.
- **Файл и строка:** app/services/parser.py, строка 43.
- **Код до:** "city_name": item.city.name.strip(),
- **Код после:**

  "city_name": item.city.name.strip() if item.city and item.city.name else “freelance”,

Итог

- Все **8** намеренно внедрённых багов найдены и исправлены.
- Приложение успешно запускается через docker compose up --build.
- Alembic создаёт таблицу, парсинг работает каждые 5 минут без ошибок.
- Swagger-документация доступна по http://localhost:8000/docs.
- Эндпоинт GET /api/v1/vacancies/ возвращает список реальных вакансий.

Selectel Vacancies API 0.1.0 OAS 3.1

/openapi.json

vacancies

- GET /api/v1/vacancies/ List Vacancies Endpoint
- POST /api/v1/vacancies/ Create Vacancy Endpoint
- GET /api/v1/vacancies/{vacancy_id} Get Vacancy Endpoint
- PUT /api/v1/vacancies/{vacancy_id} Update Vacancy Endpoint
- DELETE /api/v1/vacancies/{vacancy_id} Delete Vacancy Endpoint

parser

- POST /api/v1/parse/ Parse Endpoint

Schemas

HTTPValidationError > Expand all object

VacancyCreate > Expand all object

Рисунок 6 открыл http://localhost:8000/docs

Code | Details

200

Response body


```
[
  {
    "title": "Backend-разработчик в команду Managed Kubernetes",
    "timetable_mode_name": "Гибкий",
    "tag_name": "backend",
    "city_name": "Санкт-Петербург",
    "published_at": "2026-03-13T11:52:03.075957Z",
    "is_remote_available": true,
    "is_hot": false,
    "external_id": 1648,
    "id": 1,
    "created_at": "2026-03-15T17:37:13.311067Z"
  },
  {
    "title": "Системный администратор в команду частного облака",
    "timetable_mode_name": "Гибкий",
    "tag_name": "admin",
    "city_name": "Санкт-Петербург",
    "published_at": "2026-03-13T08:46:12.578010Z",
    "is_remote_available": true,
    "is_hot": false,
    "external_id": 1650,
    "id": 2,
    "created_at": "2026-03-15T17:37:13.311067Z"
  },
  {
    "title": "Практикант в инженерно-технический отдел",
```


Download

Response headers

```
content-length: 8597
content-type: application/json
date: Sun,15 Mar 2026 17:48:59 GMT
server: uvicorn
```

Рисунок 7 В эндпоинте GET раскроем список и нажмем Try out – Execute

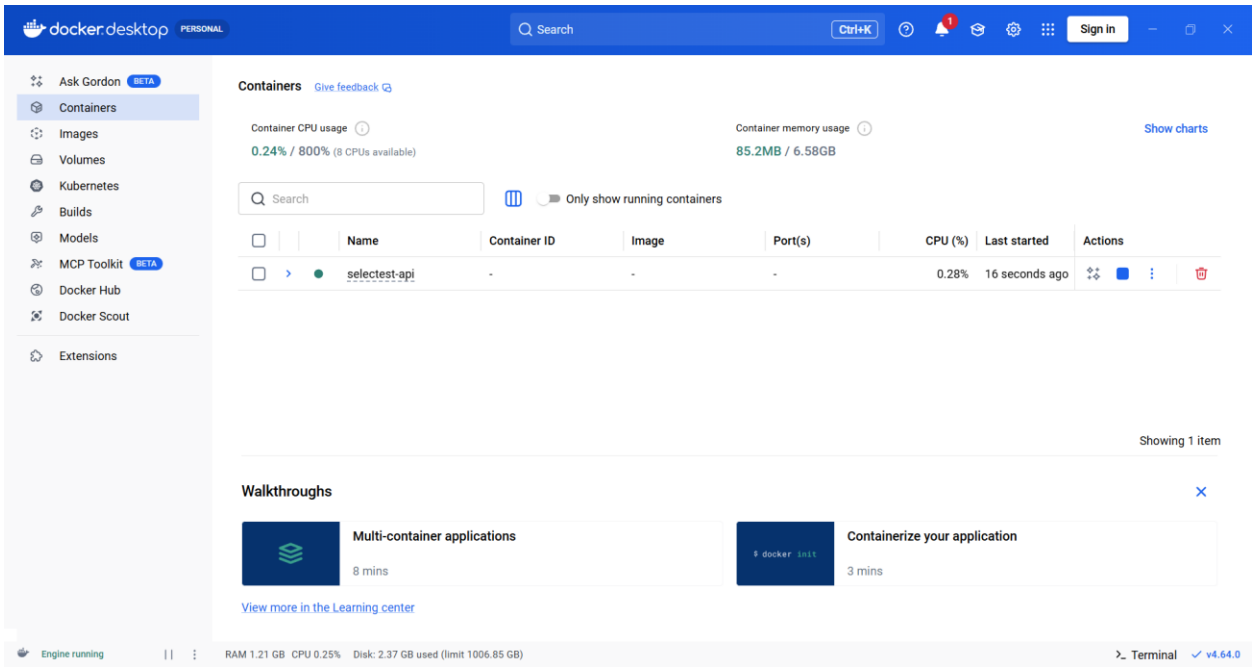

Рисунок 8 окно Docker Desktop

Проверка по критериям успешно выполненной работы:

1. Успешно запустил приложение (без ошибок) — (логи чистые, Uvicorn запущен).
2. Убедился, что парсинг вакансий выполняется корректно (без падений) — (после фикса city=None парсинг завершается успешно).
3. Обеспечил работу всех CRUD-операций — (проверено через Swagger: GET/POST/PUT/DELETE работают).
4. Убедился, что фоновая задача парсинга работает по заявленному расписанию — (APScheduler запускает парсинг каждые 5 минут, логи подтверждают).